BACCARAT MULTI-TABLE ANALYTICS AGENT

Implementation and codebase handbook

Docker Compose • Python • Playwright • OpenCV • PostgreSQL • Excel

Version 2.0 • 22 October 2026

Scope: multi-table observation and historical reporting. No automated betting or claim of a reliable next-round prediction. Demo data is synthetic.

Contents

README

01 architecture

02 installation

03 codebase

04 screen reading

05 round verification

06 data model

07 statistics and excel

08 operations

09 testing

10 production roadmap

11 faq

12 agent specification

13 integration contracts

14 acceptance

15 source survey

16 dependencies and version policy

17 source integration

Baccarat Multi-Table Analytics Agent — Docker MVP v2

Language: English. Purpose: a reproducible observation and historical reporting prototype, with a clear path toward a source-specific agent.

This project does not place bets or provide reliable next-round predictions. A screenshot or a roadmap marker does not prove that a new round has ended. Color detections remain OBSERVED_NOT_CONFIRMED. The sample CSV contains synthetic rounds, not data from a real website.

The maintained documentation is README.md, AGENTS.md, the Markdown files in docs/, and the schema notes in samples/. The combined Markdown, DOCX, and PDF handbooks are generated in English from README.md and docs/.

Version baseline: CPython 3.12.x, Playwright 1.55.0, and PostgreSQL 17. See the complete direct-dependency inventory and version policy. Several Python packages still use version ranges, and indirect packages are not locked; the current files are not yet a reproducible production dependency lock.

Target operator question: For each authorized table, between 07:00 and 21:00 local time, how many verified red-result streaks first reach 13 consecutive rounds? The read-only report-red-window command now counts this from stored confirmed rounds when the operator supplies a source-validated red-to-outcome mapping. A maximal streak is counted once when its 13th result occurs in the window; a streak of 14 is not counted twice. See docs/07-statistics-and-excel.md for boundary and gap rules. The current POC does not automatically verify results from a real site or measure complete coverage.

To rebuild the handbooks after editing their sources:

```
python3 -m pip install -r docs-requirements.txt

python3 tools/build_full_guide.py

python3 tools/build_manual_docx.py

python3 tools/build_manual_pdf.py
```

The generated files are FULL_DEVELOPER_GUIDE.md, BACCARAT_AGENT_DEVELOPER_GUIDE.docx, and BACCARAT_AGENT_DEVELOPER_GUIDE.pdf. The English demo Excel workbook is BACCARAT_Excel_Demo_Formatted.xlsx.

What works today

Capability	Current state
Generate a synthetic six-table lobby and preview configured ROIs	Implemented
Count red/blue markers in a screenshot	Implemented for the demo image; real layouts need calibration
Capture a browser page with Playwright	Implemented for accessible pages
Monitor browser frames and store marker observations	Implemented by monitor-browser; never writes confirmed rounds
Read an authorized DOM layout or structured export through a verification pipeline	Implemented as configurable code; no real website layout has been validated
Import human-reviewed CSV rounds into PostgreSQL with duplicate detection	Implemented; each CSV batch is transactional
Store observation gaps	Implemented; repeat imports are not idempotent yet
Compute historical Banker/Player/Tie counts, strict streaks, and Wilson 95% intervals	Implemented from confirmed rounds
Export a five-sheet Excel workbook	Implemented
Export a separate 07:00–21:00 Excel count of verified 13-red events	Implemented for stored confirmed rounds with an explicit red mapping; coverage remains unknown

Capability	Current state
Validate and persist structured round evidence	Connected to a transactional writer with evidence, gaps, checkpoints, and conflict quarantine; source trust still requires review
Tests	30 tests passed in Docker with PostgreSQL and Chromium; fourteen require a server, one requires Chromium, and fifteen run without either
Automatically confirm every round on a real website	Generic DOM worker exists; no named website adapter or accuracy evidence yet
Continuous operation	monitor-dom --iterations 0 loops with reconnect attempts; no production soak or managed session lifecycle yet
Automatic login, CAPTCHA bypass, private API access, or betting	Outside scope

Run the Docker demo

Requirements: Docker Desktop with WSL 2 on Windows, or Docker Engine with Compose v2 on Ubuntu. Allow about 4 GB of free RAM and several GB of disk space for Playwright and PostgreSQL. Build requires internet access. Set a unique strong POSTGRES_PASSWORD in the untracked .env file before a real deployment.

```
cp .env.example .env

docker compose build

docker compose up -d db

docker compose run --rm preview

docker compose run --rm monitor-image

docker compose run --rm import-demo

docker compose run --rm import-gaps-demo

docker compose run --rm replay-export-demo

docker compose run --rm report

docker compose run --rm test
```

In PowerShell, use Copy-Item .env.example .env. PostgreSQL data persists in the named pg_data volume; image and Excel outputs remain under output/. The demo workbook is output/reports/baccarat_daily_2026-10-19.xlsx. Run import-gaps-demo only once: the current gap table does not deduplicate. Keep demo and real data in separate databases or Compose projects.

To move a legacy SQLite database into an empty PostgreSQL database, run docker compose run --rm migrate-sqlite. The job copies a consistent SQLite snapshot in one PostgreSQL transaction and refuses a nonempty target. Back up both databases first; see docs/06-data-model.md.

See docs/02-installation.md for host setup and the exact behavior of each command.

Repository map

Path	Purpose
AGENTS.md	Instructions and work order for AI coding agents
agent/AGENTS.md; tests/AGENTS.md	Package-specific and test-specific instructions
agent/cli.py	CLI entry point
agent/browser.py	Browser capture and observation loop

Path	Purpose
agent/config.py; agent/vision.py	ROI configuration and marker detection
agent/round_validator.py	Conservative teaching FSM
agent/site_adapter.py; agent/pipeline.py	Source adapter contract and confirmed-candidate validation
agent/adapters/; agent/coordinator.py; agent/live_store.py	Structured export and DOM adapters, per-table coordination, and durable source writes
agent/storage.py; migrations/	PostgreSQL connection, transactional writes, and versioned schema
agent/analytics.py; agent/report.py	Historical metrics and Excel export
agent/red_window.py; agent/red_report.py	Verified 13-red window counting and separate Excel export
config/	Synthetic and unverified real-lobby ROI examples
assets/synthetic_lobby.png	Synthetic image for learning and tests
samples/	Synthetic verified-round CSV, gap CSV, and structured export fixtures
tests/	Unit tests
docs/	Architecture, implementation, data, operations, and acceptance criteria
tools/build_full_guide.py	Rebuilds the combined Markdown handbook
tools/build_manual_docx.py; tools/build_manual_pdf.py	Rebuild the English DOCX and PDF handbooks
tools/migrate_sqlite_to_postgres.py	One-time legacy SQLite import into empty PostgreSQL

Reading order

1. docs/01-architecture.md — system boundary and data flow.
2. docs/02-installation.md — setup and demo.
3. docs/03-codebase.md — each module and its input/output.
4. docs/04-screen-reading.md — ROI and visual calibration.
5. docs/05-round-verification.md — round identity, FSM, and duplicate prevention.
6. docs/06-data-model.md — PostgreSQL, legacy migration, CSV, and backup.
7. docs/07-statistics-and-excel.md — report definitions and limitations.
8. docs/08-operations.md — 24/7 design and scheduling.
9. docs/09-testing.md — tests and troubleshooting.
10. docs/10-production-roadmap.md — phased implementation.
11. docs/11-faq.md — common questions.
12. docs/12-agent-specification.md — requirements for the target agent.
13. docs/13-integration-contracts.md — event contracts and storage changes.
14. docs/14-acceptance.md — ground truth, metrics, and release gates.
15. docs/15-source-survey.md — survey template for an authorized source.
16. docs/16-dependencies-and-version-policy.md — Python, package inventory, and release locking policy.
17. docs/17-source-integration.md — structured export replay and configurable DOM worker.

AI coding agents should start with AGENTS.md. The source-specific adapter cannot be completed until docs/15-source-survey.md has been filled for an authorized website or export.

The source-independent pipeline can be tried with docker compose run --rm replay-export-demo. See docs/17-source-integration.md for its JSONL contract and the observation-only DOM trial. The demo uses a

separate synthetic table ID and does not validate a real website.

Trying an authorized real page

Use only a site you are permitted to observe and follow its terms. Do not put credentials, cookies, tokens, balances, or personal screenshots in the repository. The current browser command does not log in. If the page requires a session, design a permitted session mechanism without bypassing access controls.

Capture at a fixed viewport, update config/real_lobby.example.yaml for that exact screenshot, and inspect the annotated preview:

```
docker compose run --rm preview python -m agent.cli preview \

  --image assets/my_lobby.png --config config/real_lobby.example.yaml \

  --out output/my_lobby_roi.png
```

The assets mount is read-only inside Docker. Put your authorized screenshot on the host first. The current real-lobby ROI coordinates are only approximate; do not use them as validated ground truth.

A real adapter must extract a trustworthy round ID, phase, final result, and roadmap revision, then pass the acceptance tests before automatic confirmed-round writes are enabled. Never map marker counts directly to Banker or Player rounds.

Statistical limits

Reported rates describe the confirmed sample, not a betting edge. The current strict streak definition treats Tie and GAP as breaks. Wilson intervals express sampling uncertainty but do not correct missing tables, selection bias, hidden shoes, or wrong labels. The application cannot infer other players' stakes from a roadmap.

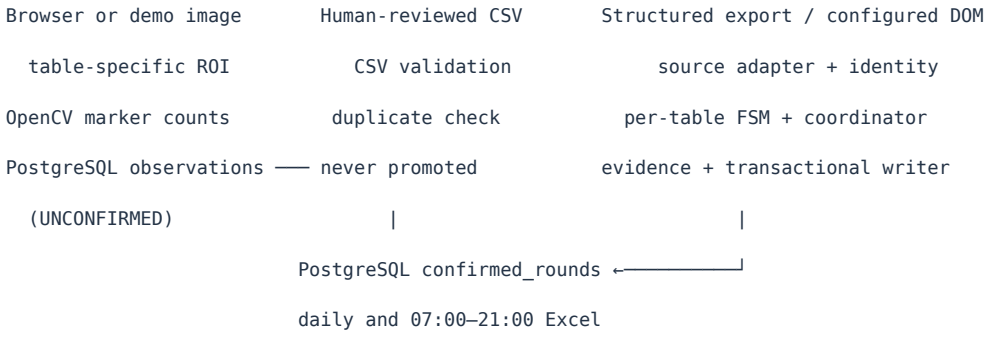
01 — Architecture and data flow

Goal

For each day, report how many tables were observed; each table's confirmed Banker, Player, and Tie rounds; its longest strict Banker/Player streak; the number of Banker streaks reaching 13; missing data; and uncertainty around historical rates. This is descriptive analytics, not a prediction or betting system.

The separate report-red-window command answers a narrower question from stored confirmed rounds: from 07:00 to 21:00 in the fixed Asia/Ho_Chi_Minh time zone, how many distinct verified red-result streaks first reach 13 consecutive rounds on each table? The mapping from a site's red display to a final outcome must be established in its source survey. The current POC cannot automatically verify a live site's rounds or prove complete coverage.

Separate evidence paths in the prototype



There is no automatic arrow from marker observations to confirmed_rounds. A separate structured source path now connects extracted evidence to the FSM, coordinator, and durable writer. Its trust flag must be enabled only after source-specific acceptance tests. No named website adapter has passed that review. See docs/12-agent-specification.md, docs/13-integration-contracts.md, and docs/17-source-integration.md.

Multiple tables

Each visible table has an ROI with x, y, width, height in raw screenshot pixels. A frame passes through every configured ROI; observations are tagged with table_id. If a table disappears or the layout changes, mark that table unknown or gapped rather than assigning another table's result to it.

A single lobby screenshot covers only visible tables. Pagination, scrolling, lazy loading, and background-tab throttling can hide rounds. Do not claim continuous coverage for tables that were not actually observed. Scaling needs permitted browser contexts or tabs, table ownership, and measured latency.

Stored data classes

Class	Meaning	Used for outcome rates?
observations	Red/blue marker counts seen in an ROI at a time	No
confirmed_rounds	One final result with a unique identity and verified source	Yes
observation_gaps	A known interruption that may break streak continuity	Used to break strict streaks

Prototype and target system

The prototype provides static-image calibration, browser capture, PostgreSQL storage, transactional reviewed CSV import, a conservative FSM, a structured source coordinator with persistent checkpoints/gaps/evidence/conflicts, and Excel output. A production system also needs an authorized session lifecycle, a validated site-specific result/ID extractor, measured coverage, evidence retention, backup, monitoring, and measured precision/recall on labeled captures.

Principal risks

- Reading the same frame hundreds of times must create zero new rounds.
- A shoe reset or roadmap redraw can look like a new round.
- Tie and pair icons can resemble result markers.
- Browser resize, zoom, theme changes, or card reordering can invalidate ROI mapping.
- Background tabs can stop rendering fresh frames.
- Full screenshots can expose account information; store minimal masked regions.
- A container restart cannot recover unseen rounds. Record a GAP and rebaseline.

02 — Installation and PostgreSQL demo

Requirements

Windows 10/11 with Docker Desktop and WSL 2, or Ubuntu with Docker Engine and Compose v2. Check docker --version, docker compose version, and docker info. Allow several GB of disk space for Playwright plus the PostgreSQL volume, and at least 4 GB of free RAM for the demo. The first build needs internet access. No Baccarat account is needed.

Use CPython 3.12.x for a host installation. The Docker application image is Playwright 1.55.0 on Ubuntu 24.04 Noble, and the database image is PostgreSQL 17. Exact Python patch and indirect package versions are not currently locked. The package-by-package inventory and release requirements are in docs/16-dependencies-and-version-policy.md.

Copy .env.example to .env and replace POSTGRES_PASSWORD with a unique password before connecting the database to anything beyond a local demo. .env is ignored by Git. The Compose database port is bound to 127.0.0.1 only.

PowerShell

```
Copy-Item .env.example .env

docker compose build

docker compose up -d db

docker compose run --rm preview

docker compose run --rm monitor-image

docker compose run --rm import-demo

docker compose run --rm import-gaps-demo

docker compose run --rm replay-export-demo

docker compose run --rm report

docker compose run --rm test
```

Open output\roi_preview.png and output\reports\baccarat_daily_2026-10-19.xlsx.

Ubuntu or WSL shell

```
cp .env.example .env

docker compose build

docker compose up -d db

docker compose run --rm preview

docker compose run --rm monitor-image

docker compose run --rm import-demo

docker compose run --rm import-gaps-demo

docker compose run --rm replay-export-demo

docker compose run --rm report

docker compose run --rm test
```

The db health check must pass before jobs that need storage start. PostgreSQL data lives in the named pg_data volume; output images and workbooks live under output/ on the host. No SQLite file is created by the new runtime.

Expected behavior

- preview loads a synthetic six-table image and writes an annotated PNG. R/B labels are marker counts, not round counts.
- monitor-image reads the same image three times and writes 18 observation rows (six tables times three frames). It writes zero confirmed rounds.
- import-demo adds 46 synthetic confirmed rows. A second import reports 46 exact duplicates and inserts zero.
- import-gaps-demo adds one synthetic gap for B07. Run it once because current gap import is not deduplicated.
- report writes five Excel sheets for REPORT_DATE=2026-10-19 from .env. Override the command for another date:

```
docker compose run --rm report python -m agent.cli report --date 2026-10-20
```

- report-red-window writes a separate Excel workbook for verified red-result runs first reaching 13 between 07:00 and 21:00 local time. Confirm the site's red mapping before choosing BANKER or PLAYER:

```
docker compose run --rm report python -m agent.cli report-red-window \
  --date 2026-10-19 --source DEMO_FIXTURE --red-outcome BANKER
```

This command reads confirmed rounds only. It marks observation coverage UNKNOWN and cannot establish that no rounds were missed.

- replay-export-demo processes four synthetic structured frames and stores one evidence-backed confirmed round on first run. Keep its data separate from real sources.
- test runs 30 tests: fourteen PostgreSQL integration tests in isolated temporary schemas, one Chromium DOM-reader smoke test, and fifteen source-independent tests. The server must be healthy.

Keep synthetic and real data in separate databases or Compose projects.

Inspect, back up, and reset

Inspect counts without exposing the database to the network:

```
docker compose exec db psql -U baccarat -d baccarat -c &amp;quot;SELECT COUNT(*) FROM confirmed_rounds&amp;quot;;
```

Replace the user/database names if you changed POSTGRES_USER or POSTGRES_DB. For backups, use pg_dump and verify a restore; see docs/06-data-model.md. docker compose down leaves the named volume intact. docker compose down -v deletes the PostgreSQL volume and is appropriate only for a disposable demo after a verified backup. Do not run it against production data.

Legacy SQLite import

If you have data/agent.sqlite3 from the old version, back it up and use a fresh empty PostgreSQL database:

```
docker compose run --rm migrate-sqlite
```

The tool takes a consistent read-only snapshot of the SQLite file and imports observations, confirmed rounds, and gaps in one PostgreSQL transaction. It refuses a nonempty target, preventing a partial merge. See docs/06-data-model.md for validation steps.

Host Python without Docker

Use a Python 3.12 virtual environment, install Python dependencies from requirements.txt, and provide a PostgreSQL 17 server. Set PGHOST, PGPORT, PGDATABASE, PGUSER, and PGPASSWORD, or DATABASE_URL. For the fourteen database integration tests also set RUN_PG_TESTS=1. Set RUN_BROWSER_TESTS=1 only when Playwright Chromium and its system libraries are installed. Without either dependency, those fifteen tests skip and the other fifteen still run. Compose is the documented runtime path and enables both integration groups. The current version ranges mean a host install may differ from another install; see docs/16-dependencies-and-version-policy.md before treating it as a reproducible release.

For the structured export and DOM source path, see docs/17-source-integration.md. config/real_dom.example.yaml contains placeholder selectors and https://example.com; it is not a ready Baccarat source.

03 — Codebase walkthrough

agent/cli.py

argparse provides preview, monitor-image, capture-browser, monitor-browser, monitor-dom, replay-export, ingest-csv, ingest-gaps, report, and report-red-window. Docker Compose reuses the same image with different commands. The CLI never logs in or places bets.

Command	Input	Output	Writes confirmed rounds?
preview	Image and ROI YAML	Annotated image and JSON marker counts	No
monitor-image	Static image and iteration count	PostgreSQL observations	No
capture-browser	URL and viewport	Screenshot	No
monitor-browser	URL, ROI, iterations	Annotated frames and observations	No
monitor-dom	Reviewed DOM selector configuration, interval, optional trusted-ID gate	Cropped table evidence, source gaps, and candidates when trusted	Conditionally, after explicit trust gate
replay-export	Structured JSONL source states and explicit source trust gate	Source events, evidence, gaps, checkpoints	Conditionally, after explicit trust gate
ingest-csv	Reviewed CSV	PostgreSQL confirmed_rounds	Yes
ingest-gaps	Gap CSV	PostgreSQL observation_gaps	No
report	Date and PostgreSQL database	Five-sheet Excel workbook	No
report-red-window	Date, source, explicit RED mapping, and PostgreSQL database	Three-sheet 07:00–21:00 reached-13 Excel workbook	No

Use python -m agent.cli <command> --help for CLI options.

agent/config.py and agent/vision.py

config.py parses a tables list of table IDs and pixel rectangles. Its validation catches negative or out-of-frame ROIs, but an ROI can still point at the wrong table while remaining inside the screenshot. Always inspect the annotated preview.

vision.py loads images as BGR, converts crops to HSV, thresholds red/blue, filters contours by area and circularity, and returns markers and per-table counts. Its thresholds were chosen for the synthetic image. Marker count must never be written as Banker/Player round count.

agent/browser.py

capture_page opens an internal Playwright Chromium page, waits for DOM content and a fixed delay, then saves one screenshot. monitor_browser opens one page and captures N frames; each is decoded, analyzed by ROI, and written as marker observations. monitor_dom_browser uses a configurable DOM reader, cropped table evidence, and the source coordinator. It can loop when iterations is zero, records errors as gaps, and retries navigation. It has no persistent login/session workflow or validated named Baccarat layout.

agent/round_validator.py

TableEvidence contains table_id, frame_id, phase, source_round_id, final_result, roadmap_revision, and round_id_trusted. ConservativeRoundValidator is a per-table teaching FSM. It requires BETTING → DEALING → two independent matching RESULT frames, a trusted round ID, and a changed roadmap revision. It returns VerifiedResult or None. It does not read screenshots, persist state, or write the database.

agent/site_adapter.py and agent/pipeline.py

site_adapter.py defines CapturedFrame, ExtractedEvidence, and the SiteAdapter protocol. adapters/structured.py reads explicit source states and defaults IDs to untrusted. adapters/dom.py reads configured DOM text fields and table-cropped screenshots; the shipped selectors are placeholders. pipeline.py runs the FSM for one explicitly selected source and returns ConfirmedCandidate with two evidence references and a time policy. coordinator.py validates source/table identity, frame order, restart and gap behavior, then passes candidates to live_store.py. The coordinator does not certify that a source adapter is accurate; a reviewed corpus and trust gate are still required.

agent/storage.py

connect opens PostgreSQL through Psycopg and applies numbered migrations under an advisory lock. Migration 001 creates the original schema; migration 002 adds per-table time indexes; migration 003 adds source evidence, idempotent source gaps, conflicts, processed frames, and checkpoints. record_observations writes only OBSERVED_NOT_CONFIRMED rows. insert_verified_round uses a transaction and ON CONFLICT to deduplicate safely across concurrent writers; a conflicting payload raises an error. live_store.py retains conflicting source payloads for review. The old manual mark_gap path still does not deduplicate repeats. See docs/06-data-model.md for schema, backups, and legacy migration.

agent/analytics.py and agent/report.py

analytics.py converts the selected Asia/Ho_Chi_Minh day to a half-open UTC interval, reads confirmed rounds and gaps, and computes counts, strict streaks, Wilson 95% intervals, and observation frame counts. Tie/GAP breaks a strict streak; the implementation calculates streaks within the selected day.

report.py writes Summary, Rounds, Streaks, Quality, and Methodology sheets through XlsxWriter. It warns when any row comes from DEMO_FIXTURE. Keep real and synthetic databases separate.

Tools, tests, and deployment files

agent/red_window.py counts one event when a verified strict red run first reaches 13 in the local 07:00–21:00 window, using up to 13 earlier events to handle a run crossing 07:00. agent/red_report.py exports the separate Excel workbook, including source evidence references when available. The caller must supply a source and BANKER or PLAYER as its validated red mapping; the workbook marks coverage UNKNOWN. A table ID shared by multiple sources is rejected until the schema gains a source namespace. tools/generate_synthetic_lobby.py creates a harmless six-table learning image. tools/build_full_guide.py combines maintained Markdown files. tools/migrate_sqlite_to_postgres.py imports a legacy SQLite snapshot into an empty PostgreSQL database. The test suite has 30 tests: fourteen PostgreSQL integration tests, one Chromium reader smoke test, and fifteen source-independent tests. Dockerfile packages Python, Psycopg, migrations, and Playwright Chromium. compose.yaml adds a PostgreSQL service with a named volume and health check; data/ is mounted read-only only for legacy import.

Example call paths

```
docker compose run --rm preview

  → agent.cli preview

  → config.load_regions
```

```
  → vision.load_image / analyze_frame
  → output/roi_preview.png and JSON

docker compose run --rm report
  → agent.cli report
  → storage.connect
  → report.export_daily
  → analytics.daily_data
  → output/reports/baccarat_daily_YYYY-MM-DD.xlsx
```

Read CLI → config → vision → browser → validator → adapter contract → pipeline → storage → analytics → report while exploring the code.

04 — Reading the screen and calibrating ROIs

Understand the display

A table video or status panel may show BETTING, DEALING, RESULT, a clock, a final outcome, or a round ID. A roadmap is historical display data. Bead plates, big road, and other roads differ in ordering and in how they show Tie, pairs, and streaks. A colored marker is not always one independent completed round.

config/real_lobby.example.yaml contains approximate roadmap coordinates from an old 1536×864 screenshot. They have not been independently validated.

ROI coordinates

A Region of Interest is a rectangle (x, y, width, height) measured in screenshot pixels from the upper-left corner. Assign one ROI to each visible table. Avoid dealer video, user names, and balances to reduce noise and exposure of personal data.

Calibrate an authorized screenshot

1. Choose a fixed viewport, scale factor, and browser zoom.
2. Capture a page you are allowed to view. Mask account details before storing or sharing the image.
3. Measure each table's roadmap rectangle with an image editor.
4. Update config/real_lobby.example.yaml using the table's displayed identity, not its card position.
5. Put the screenshot on the host as assets/my_lobby.png and run:

```
docker compose run --rm preview python -m agent.cli preview \
  --image assets/my_lobby.png --config config/real_lobby.example.yaml \
  --out output/my_lobby_roi.png
```

6. Inspect output/my_lobby_roi.png. Every box must cover the correct table roadmap. Compare highlighted markers with the image, then adjust HSV/contour thresholds only with labeled examples.
7. Keep the masked image and the matching configuration version for reproducibility.

ROI bounds validation alone cannot detect a box that has shifted onto the wrong but still in-bounds card.

Common visual failure modes

Tie can overlay another marker; pair symbols can share a result color; thin rings can fragment; markers can merge; themes and anti-aliasing alter HSV; a full roadmap can scroll or redraw. Label at least 100 varied images per ROI for marker-level evaluation, then measure false positives and negatives. Accuracy on the synthetic image is not evidence of accuracy on a real site.

Detecting a new round

One screenshot is a state, not an event. A trusted source round ID and published final result, or a tested phase sequence plus a verified roadmap transition, are required. The prototype can validate pre-extracted TableEvidence but cannot extract these signals from a real page.

Browser capture

APP_URL defaults to https://example.com. Set only an authorized URL, then run docker compose run --rm capture-browser. To use a specific viewport:

```
docker compose run --rm capture-browser python -m agent.cli capture-browser \
  --url https://example.com --width 1536 --height 864 \
  --out output/browser_1536.png
```

Container Chromium does not see an already-open Windows Chrome window. A login page in the screenshot means this prototype has not obtained a permitted session. Do not bypass login or CAPTCHA.

05 — Round verification and duplicate prevention

The core problem

The agent needs an event meaning “round R120 has finished with Banker,” not a count of colored dots visible now. The prototype proves the reporting path with reviewed CSV and a synthetic structured export. Its FSM accepts already extracted evidence; a configurable DOM reader is wired in, but no named real page has been validated.

Current teaching FSM and target workflow

agent/round_validator.py implements UNKNOWN → BETTING → DEALING → RESULT_PENDING → DONE. It requires a trusted round ID, changed roadmap revision, and two separate matching RESULT frames. It does not parse UI, persist state, or write PostgreSQL.

The structured coordinator now records UNKNOWN/GAP, rebaselines after restart or capture gaps, writes durable evidence, quarantines conflicts, and deduplicates processed frames. Real-source health, coverage, and layout accuracy still need validation.

Example

- 12:00:00: B51 is BETTING with trusted source ID R120 and baseline revision A.
- 12:00:12: DEALING for R120. Write no round.
- 12:00:25: RESULT Banker with revision B. Hold it as a candidate.
- 12:00:30: An independent frame again shows R120, Banker, revision B. The FSM emits one VerifiedResult.
- The durable source writer records one B51/R120 round only after evidence and time checks. Reading the same result repeatedly or advancing to R121 must not create another R120 row.

This is a workflow example, not a claim about a particular site's display timing.

Evidence levels

A, preferred: published final result plus trustworthy source round ID from the UI or a permitted official export, with consistent roadmap change.

B, requiring separate validation: a complete phase sequence and a tested roadmap mapping with no observation gap, using a persistent shoe/session-local sequence ID.

C, unconfirmed: marker count, pixel differences, screenshot timestamps, resize, or hash changes alone. Keep these as observations.

Invariants

1. One frame is not one round.
2. Confirmed records require a unique key per table and source/shoe namespace.
3. Missing or contradictory evidence remains UNKNOWN or conflict.
4. Store timezone-aware UTC event times; report by the documented local day.
5. GAP interrupts strict streaks.
6. Tables are independent; ROI position is not table identity.
7. Same key with conflicting results must be quarantined, never overwritten.
8. Restart while RESULT is displayed must not insert the previous round again.

Edge cases

Tie may be an overlay; it still breaks the current strict streak definition. Roadmap reset or shoe change requires a new namespace or rebaseline. Network loss, stale images, logout, table reordering, scrolling, and lazy loading block confirmation for affected tables.

Integration steps

Configure and validate an authorized source adapter that produces TableEvidence with a separate envelope for UTC capture time, source ID, and evidence reference. agent/site_adapter.py defines the contract; agent/pipeline.py returns a ConfirmedCandidate; agent/coordinator.py and agent/live_store.py handle ordered frames, GAP persistence, evidence, duplicate keys, and conflicts. The source's layout, page freshness, ID reset rule, and final result semantics must pass the held-out checks before enabling trusted round IDs. See docs/13-integration-contracts.md, docs/14-acceptance.md, and docs/17-source-integration.md.

06 — PostgreSQL data model and legacy migration

Why PostgreSQL now?

The runtime uses PostgreSQL 17 as its default store, with a persistent named Docker volume and a health check. It supports concurrent connections, server-side transactions, and a clear path toward several workers and readers. This database change does not make the page parser or live round validator production-ready.

The Python driver is Psycopg 3. Connection parameters come from DATABASE_URL or the standard PGHOST, PGPORT, PGDATABASE, PGUSER, and PGPASSWORD environment variables. Compose sets PG* values internally, so credentials are not assembled into a URL in command output. The Compose port is bound only to the local host.

Versioned schema

migrations/001_initial.sql creates the original tables. migrations/002_red_window_lookup.sql adds descending (table_id, timestamp, id) indexes for historical per-table red-window lookups. migrations/003_verified_source_pipeline.sql adds the structured source pipeline's audit and recovery tables. Numbered migrations are applied once under an advisory lock; schema_migrations records each version and SHA-256 checksum. Applied files are checked for unexpected edits. Do not edit an already deployed migration. Future changes need new numbered migrations and rollback/restore tests.

Table	Current fields and constraints
observations	identity id, table_id, observed_at_utc TIMESTAMPTZ, nonnegative red/blue counts, OBSERVED_NOT_CONFIRMED status, evidence_path
confirmed_rounds	identity id, table_id, round_id, finished_at_utc TIMESTAMPTZ, Banker/Player/Tie result, method, source, reviewer, recorded_at_utc TIMESTAMPTZ; unique (table_id, round_id)
observation_gaps	identity id, table_id, detected_at_utc TIMESTAMPTZ, reason
schema_migrations	Applied schema version, file checksum, and UTC time
source_round_evidence	Source/table/source-round key, stored round key, two frame IDs and evidence references, finish-time policy
source_conflicts	Existing and incoming JSON payloads, unique conflict event key, OPEN review state
source_gaps	Source/table/event key, time, reason; idempotent source gap events
processed_source_frames	Unique source/table/frame ID and capture time for replay prevention
worker_checkpoints	Last processed frame ID and capture time per source/table

Time/table indexes support daily reports, and table/time indexes support bounded lookback for the 07:00–21:00 red-window report. PostgreSQL stores true timestamp values; analytics converts them to UTC ISO-8601 strings for the workbook. Both reports read their data in one read-only repeatable-read snapshot.

The legacy key (table_id, round_id) is retained for compatibility with old data. The structured source writer stores a length-prefixed source key as round_id, and the structured adapter includes shoe identity when IDs reset per shoe. The red report still rejects a table ID shared across sources because legacy gaps are not source-scoped. Before combining such providers in one database, introduce an explicit source/shoe namespace in a new migration and update reports and adapters accordingly. Manual gap import still lacks deduplication; do not import the same gaps twice. Structured source gaps use unique event keys.

CSV import and transactions

Required columns: table_id, round_id, finished_at_utc, result, method, source, reviewer, verified. verified must be YES. Timestamps need an offset. Each CSV batch is now one PostgreSQL transaction: a bad row rolls back every row in that batch. A concurrent duplicate is resolved by INSERT ... ON CONFLICT DO NOTHING; the existing payload is then compared and a conflict raises an error without overwriting it.

```
docker compose run --rm import-demo

docker compose run --rm import-demo python -m agent.cli ingest-csv \

  --input samples/my_reviewed_rounds.csv
```

The second demo import inserts zero confirmed rows. MANUAL_VERIFIED requires actual reviewed evidence for real data. DEMO_FIXTURE is synthetic and must never be presented as a website observation.

One-time import from SQLite

The old data/agent.sqlite3 is not opened by the new runtime. To migrate it:

1. Stop old SQLite writers and make a backup of the file and any WAL/SHM state. Back up the new PostgreSQL database if it already exists.
2. Start a fresh, empty PostgreSQL database with the new Compose stack. Keep the old file under data/ on the host; Compose mounts data/ read-only to the migration job.
3. Run docker compose run --rm migrate-sqlite. The tool creates a consistent SQLite backup snapshot in memory, locks the three PostgreSQL target tables, checks that they are empty, copies rows in batches, and verifies counts before committing.
4. Query counts in PostgreSQL and generate a report for a known old date. Compare confirmed outcomes, gaps, and report totals with the old export.
5. Retain the original SQLite backup until the PostgreSQL restore and report checks pass.

The migration refuses a nonempty target and a second run. An error rolls back the PostgreSQL import. It does not overwrite existing data or merge demo and real datasets.

Structured source writes

SourceCoordinator records frame identity, validates ordering against a checkpoint, resets its FSM on restart or capture gaps, and writes a candidate plus two evidence references in one transaction through live_store.py. Exact frame replay is a no-op. A conflicting round payload is retained in source_conflicts and does not overwrite the first confirmed row. A gap is inserted into both source_gaps and the legacy observation_gaps streak-break table only once per source event key. The source ID, ID/reset rule, final result semantics, and DOM selectors still require independent validation; see docs/17-source-integration.md.

Inspect and back up

Use psql inside the db container, substituting your user/database names:

```
docker compose exec db psql -U baccarat -d baccarat \

  -c &amp;quot;SELECT table_id, result, COUNT(*) FROM confirmed_rounds GROUP BY table_id, result&amp;quot;
```

A logical backup example:

```
docker compose exec -T db pg_dump -U baccarat -d baccarat -Fc &amp;gt; baccarat.backup
```

Restore into a separate database first, then compare row counts and a sample report. Protect backup files because they may contain source metadata and evidence paths. Retain screenshots only as long as required by the source policy. The named volume survives docker compose down; down -v deletes it.

Time and identity

Store event timestamps as TIMESTAMPTZ in UTC. The daily report selects the Asia/Ho_Chi_Minh day using the half-open interval [start, end), so a boundary instant appears in one report only. Its strict streaks reset at the report-day boundary. The separate red-window report uses local [07:00, 21:00), reads up to 13 prior per-table events to determine continuity at 07:00, and treats Tie or GAP as a break. Captured frame time, source finish time, and database recording time are distinct.

References

- Psycopg connection and transaction documentation
- PostgreSQL INSERT and ON CONFLICT
- Docker PostgreSQL guide
- PostgreSQL backup and restore

07 — Excel and historical statistics

Five workbook sheets

Sheet	Purpose
Summary	Per-table confirmed totals, Banker/Player/Tie, Banker rate and Wilson interval, strict streaks, gaps, observed frames
Rounds	Each confirmed round with ID, UTC time, result, method, source, reviewer
Streaks	Strict runs with result, length, start/end, and whether Banker reached 13
Quality	Recorded observation gaps and warnings
Methodology	Time zone, synthetic-data warning, Tie/GAP rules, and statistical assumptions

The workbook is output/reports/baccarat_daily_YYYY-MM-DD.xlsx. If any row uses DEMO_FIXTURE, the report marks the day as synthetic. Keep demo and real data in separate databases.

Definitions

Observed Banker share is confirmed Banker rounds divided by all confirmed Banker, Player, and Tie rounds. For zero confirmed rounds, the rate and interval stay blank rather than implying 0%.

A strict streak is consecutive results of one side. Player, Tie, or GAP ends a Banker streak; Banker, Tie, or GAP ends a Player streak. A maximal Banker run of length at least 13 counts as one reached-13 streak, not as every overlapping 13-round window. The implementation computes streaks within the selected report day.

For k Banker rounds among n confirmed rounds, Wilson 95% uses z=1.959963984540054:

```
p = k/n

denominator = 1 + z²/n

center = (p + z²/(2n)) / denominator

half_width = z * sqrt(p(1-p)/n + z²/(4n²)) / denominator

interval = [center-half_width, center+half_width]
```

The interval describes sampling uncertainty under its model. It does not repair selection bias, missed rounds, wrong labels, or unknown shoe composition.

Interpretation

A run of 13 Banker outcomes does not make Player the guaranteed next outcome. The original guide cites approximate standard eight-deck starting-shoe probabilities of Banker 45.86%, Player 44.62%, and Tie 9.52%; exact rules and remaining cards matter. A screen history alone does not establish a betting edge. Do not issue betting signals from streaks.

The synthetic CSV includes a B51 run of 13 Banker rounds, followed by Tie, then Banker and Player. The report gives max_banker_strict=13 and streak_13_banker=1.

Target report: red streaks from 07:00 to 21:00

This is the requested operational metric. It is implemented by the separate report-red-window command for stored confirmed rounds, not by the current daily workbook. The existing streak_13_banker is a whole-day

Banker metric; it cannot be presented as the requested red count without a verified source-specific color mapping and time-window calculation.

- Configure one local time zone, initially Asia/Ho_Chi_Minh, and use the half-open interval [07:00, 21:00) for each calendar date. A result at 07:00 is in scope; a result at 21:00 is outside it. Store and compare UTC finish times, then convert for display.
- Configure RED only after the authorized source survey confirms which published final outcome its red marker represents. Keep the source's final BANKER, PLAYER, or TIE label as the stored truth. Marker counts or color alone never create a round.
- Process verified rounds independently for each stable source/table identity, in result order. Opposite-side results, Tie, a known GAP, an untrusted ID, or loss of continuity end the current strict red run. A new shoe does not automatically join two runs; follow the verified source continuity rule.
- Maintain the run across 07:00 and 21:00 so boundary handling is honest. The requested count increases once when the 13th verified red result in a run finishes inside the window. A run of 14 or 20 does not produce another count. If its 13th result occurred before 07:00, continued red results inside the window do not count it again. A run reaching 13 at or after 21:00 belongs outside that day's window.
- Report the per-table count and a sum across tables, plus each counted run's source/table, first and 13th round IDs, local and UTC timestamps, and evidence references when the source pipeline stored them. The sum counts table-level events; it never merges results from different tables. The current report shows confirmed-round and recorded-gap counts, but marks coverage UNKNOWN because no independently measured continuous-source coverage ledger exists yet. Therefore its count is observed verified events, not a guaranteed complete total.
- A future exactly_13_completed_runs statistic would qualify a maximal run only when it ends at length 13. This differs from the requested reached_13_events, which includes longer runs. A run whose end is unobserved cannot yet be classified as exactly 13.

Worked example for one table: 12 verified red results before 07:00 followed by one at 07:05 produce one reached_13_events in the window. If that run continues to 14, it still produces one event and zero exactly_13_completed_runs. A separate run reaches its 13th red at 20:59 and adds one event. A third run reaches 13 at 21:00 and adds none. The target count for this table is 2, assuming no gaps or uncertain rounds.

The exactly_13_completed_runs statistic is a future extension; it is not exported now. The current database also cannot prove a run's continuity across an unrecorded outage or shoe reset. Verify the source's red mapping and data provenance before using the report.

Run the separate workbook after importing trusted, reviewed rounds:

```
docker compose run --rm report python -m agent.cli report-red-window \
  --date 2026-10-19 --source DEMO_FIXTURE --red-outcome BANKER
```

Choose PLAYER instead only if the surveyed website maps red to the published Player result. Set --source to the exact source value stored with the reviewed rounds. The command writes output/reports/baccarat_red_13_YYYY-MM-DD_07-21_<source-hash>.xlsx with Summary, Reached 13 events, and Methodology sheets. The hash distinguishes reports from different sources without placing their names in filenames. Marker observations are excluded. The current schema cannot safely combine two sources that reuse a table ID, so the report rejects that case. Never treat the UNKNOWN coverage status as a complete census.

Report another day

```
docker compose run --rm report python -m agent.cli report --date 2026-10-19
```

Without --date, Python chooses yesterday in Asia/Ho_Chi_Minh. The default Compose report command explicitly passes the sample REPORT_DATE instead; override it as above for another day.

08 — Continuous operation and daily scheduling

What exists now

Compose provides a persistent PostgreSQL db service and on-demand jobs: preview, monitor-image, capture-browser, monitor-browser, monitor-dom, replay-export-demo, import-demo, import-gaps-demo, report, test, and migrate-sqlite. monitor-browser --iterations N exits after N frames. The configured DOM worker accepts --iterations 0 for continuous capture and reconnect attempts, but its placeholder selectors are not a validated source and it has not passed a production soak test.

A production worker still needs measured reconnect/backoff, heartbeat and health status, permitted session handling, verified page freshness, rate limits, retention, graceful shutdown, a clear table owner, and operational metrics. The structured coordinator now has per-table checkpoints and GAP records; a restart cannot recover rounds unseen during downtime.

Browser loop trial

Use only an authorized page with ROIs calibrated to its viewport. For a technical loop trial:

```
docker compose run --rm monitor-browser python -m agent.cli monitor-browser \

  --url https://example.com --width 1280 --height 720 \

  --config config/synthetic_tables.yaml \

  --iterations 60 --interval 5
```

example.com contains no Baccarat data. Marker readings from it are meaningless as game results. Limit and delete screenshots that contain personal account information.

For a reviewed DOM layout, use the observation-only monitor-dom trial in docs/17-source-integration.md. Crop and retain only the permitted table evidence needed for an audit. Do not enable trusted round IDs on the placeholder config.

Daily report scheduling

On a host whose local clock is Asia/Ho_Chi_Minh, an example cron entry runs at 00:05 and allows Python to select the prior local day:

```
5 0 * * * cd /opt/Baccarat_Agent_Implementation_v2 &amp;&amp; /usr/bin/docker compose run --rm report python -m
 agent.cli report &gt;&gt; /var/log/baccarat-report.log 2&gt;&amp;1
```

Verify the host timezone and Docker path. Cron implementations differ in TZ handling. On a UTC host, 17:05 UTC is 00:05 the next day in Ho Chi Minh City; verify the selected date at the actual job time. A scheduler container is another option. Kubernetes is unnecessary at prototype scale.

Running a report again currently overwrites the same workbook. Automatic email or chat delivery needs report_runs with date, watermark, checksum, generated/sent state, and idempotent retry. This prototype does not send reports.

Scaling tables

Partition table ownership across workers only after measuring real CPU, RAM, browser contexts, render latency, and coverage. PostgreSQL is the default shared store; use connection limits or pooling after benchmarking, and keep table ownership explicit. A worker needs a heartbeat, backlog, last observation age, and last confirmed age. Stale frames create UNKNOWN/GAP, not inferred rounds.

Security and resilience

Use a session only where authorized. Keep the PostgreSQL password in untracked .env or a deployment secret store, not committed Compose files; do not publish credentials or cookies in logs. Do not bypass CAPTCHA or anti-bot controls. Save the smallest useful masked evidence, apply retention limits, make pg_dump or physical backups, and test restoration. Benchmark before setting worker resource limits.

09 — Tests and troubleshooting

Required scenarios

ID	Scenario	Expected result
T01	Preview the synthetic six-table image	ROIs align and visible marker counts are nonzero
T02	Capture the same image 100 times	More observations; zero new confirmed rounds
T03	Import the 46-round CSV twice	First import +46; second +0 with duplicates reported
T04	Reuse one round ID with a different result	Conflict; existing row remains unchanged
T05	13 Banker, then Tie, then Banker	Maximum strict Banker run 13; one run reaches 13
T06	Banker → GAP → Banker	Two separate runs
T07	B51 Banker and B20 Player	Independent table summaries
T08	Local 23:59 and 00:00 boundary	Correct day assignment, no double count
T09	Page logged out	No confirmed outcome; quality warning in target implementation
T10	Viewport, zoom, or card position changes	Block wrong ROI mapping
T11	Network outage	GAP; no streak across unknown interval
T12	Restart while RESULT remains visible	No duplicate confirmed round
T13	Tie overlay	No extra round from a secondary marker
T14	Shoe change or roadmap reset	Rebaseline before confirmation
T15	Same source key with conflicting data	Quarantine; no silent correction

The 30 current tests and CLI demo cover only parts of T01–T08, PostgreSQL storage behavior, legacy migration, report export, red-window boundary and gap rules, mixed-source rejection, a synthetic structured candidate-to-storage flow, and a Chromium DOM-reader smoke test. Fourteen PostgreSQL integration tests run when RUN_PG_TESTS=1 and one browser smoke test runs when RUN_BROWSER_TESTS=1; Compose supplies both. T09–T15 still need a validated named-source adapter, real labeled evidence, and held-out integration tests. The full acceptance matrix is in docs/14-acceptance.md.

Commands

```
python3 -m unittest discover -s tests -v

docker compose run --rm test
```

To observe import deduplication, run docker compose run --rm import-demo twice on a disposable demo database. The second run should print inserted=0 and exact_duplicates=46. Avoid running import-gaps-demo twice.

Ground truth for a real UI

Start with 200–500 ended rounds across 3–6 tables and varied phases, outcomes, layouts, resets, and outages, subject to access and privacy rules. Two people label table ID, source round ID, final outcome, timing, and gaps independently. Resolve disagreements against evidence. Split calibration and held-out sets by session/day. Report precision, recall, duplicates, missed rounds, coverage, and gap detection per table. A proposed release

target is at least 99% confirmation precision and zero duplicate confirmed rows; those are criteria, not measured prototype results.

Troubleshooting

- Docker daemon unavailable: start Docker Desktop/Engine and check user permissions.
- PostgreSQL unhealthy or connection refused: check docker compose ps db, POSTGRES_* values in .env, and whether the named volume was initialized with different credentials. Do not delete a non-demo volume to fix a login error.
- Image pull or build fails: check registry access, network, and Playwright image tag.
- No Excel output: check the date, imported confirmed data, output mount, and write permissions. Observations alone produce no rounds.
- Empty workbook: the chosen day has no confirmed rounds, or only observations.
- ROI shifted: match screenshot dimensions, zoom, and config; inspect the overlay manually.
- Wrong marker counts: recalibrate HSV/contours with labeled real images; inspect Tie/pair icons.
- CONFLICT on import: review the original evidence; do not manually overwrite the database.
- Login screenshot: the current browser has no session; do not bypass access controls.
- Time appears seven hours off: distinguish UTC storage from Asia/Ho_Chi_Minh report time.
- Import error for cv2/numpy/yaml: install requirements.txt; it is an environment problem, not a passed business test.

In the rebuilt Docker image, all 30 tests passed against PostgreSQL 17 and Chromium. Without a running database or Chromium, fifteen integration tests skip and the other fifteen run. docker compose run --rm test enables both integration groups on a machine with Docker. The generated preview, structured export, and sample Excel are synthetic; real-source accuracy still needs separate evaluation.

10 — Roadmap from demo to a multi-table system

The detailed target requirements, integration contract, acceptance plan, and source questionnaire are in chapters 12–15. Start source-specific work only after an authorized source has been surveyed.

Before a production-oriented release, complete the Python and container version controls in docs/16-dependencies-and-version-policy.md. The current POC has no committed transitive dependency lock or immutable image digest.

Phase 0 — Understand the prototype

Run preview and inspect its ROIs. Run monitor-image and verify that confirmed rounds do not increase. Import synthetic CSV, generate Excel, and read the code. Exit condition: the team can explain the difference between observations and confirmed rounds.

Phase 1 — One source, then 3–6 tables

Fix viewport and table identity. Create labeled image/video evidence covering phases, Banker, Player, Tie, reset, and disconnect. Configure or implement and validate a source adapter and round-ID trust rules. The generic FSM, coordinator, evidence persistence, conflict review, and restart rebaseline are implemented; prove they work for the selected source and layout. Measure precision, recall, missed rounds, duplicates, and coverage on held-out sessions. Exit condition: the metrics meet predeclared per-table thresholds and every confirmed result has evidence.

Confirm what the source's red marker means before using the existing 07:00–21:00 report on real data. The report now includes source evidence references when available; extend it with measured coverage after verified rounds and gap handling are reliable. Validate its boundary, per-table, and reached-13 cases A13–A16.

Phase 2 — Continuous Compose operation

Harden the existing DOM worker loop with measured backoff, heartbeat, permitted session lifecycle, page freshness checks, rate limits, and graceful shutdown. Retain GAP logging and checkpoints. Use the existing PostgreSQL store with bounded connections; back up and restore it; rotate logs and retain minimal screenshots. Schedule an idempotent daily report. Run a 24–72 hour soak test including planned outages and restarts. Exit condition: downtime and unknown coverage are visible and state survives restart.

Phase 3 — Larger deployments

Verify that the source and infrastructure actually allow simultaneous access to all target tables. Benchmark browser sessions, CPU, memory, tab throttling, database connections, and frame latency while scaling gradually. Assign one worker owner per table group and introduce a queue only when measured load requires it. Add a read-only dashboard for coverage, freshness, delays, confirmed, and unknown. Test restarts, network loss, table reordering, and roadmap resets. Exit condition: every table meets its quality and coverage targets.

Definition of done

- Each confirmed round has a trustworthy key and evidence; marker counts alone never confirm rounds.
- Duplicate and conflict handling survive restart and concurrent processing.
- GAP, UNKNOWN, and low coverage appear in reports.
- Excel documents time zone, provenance, methodology, and synthetic data.
- Results are checked against held-out ground truth, not only the synthetic demo.
- No betting automation or unsupported probability claim is presented.

Decisions to record

Decision	Why it matters
Lobby screenshot or individual table tabs	Coverage and browser cost
Stable source round ID and shoe namespace	Deduplication and recovery
Roadmap type and Tie semantics	Correct outcome interpretation
Permission for automated observation	Access and acceptable rate
Evidence retention period	Privacy and storage cost
Required statistics	Scope and report design

Reference links: Playwright Docker, OpenCV contours, Docker Compose, Psycopg 3, PostgreSQL backup, Python zoneinfo, Wizard of Odds Baccarat basics.

11 — Frequently asked questions

Can a Docker container see my open Windows Chrome window? No. Playwright starts Chromium inside the container. A normal container cannot capture the Windows desktop merely because Chrome is open there. The prototype can preview a host-provided image or capture an accessible page in its own browser.

Why do 20 red markers not mean 20 Banker rounds? Roadmaps can use different layouts, overlays, and scrolling. A static image does not identify which marker is new or whether a round has ended.

Can machine learning predict the next red or blue result? Historical patterns can be studied, but this project does not treat them as a reliable betting advantage. It reports descriptive statistics and uncertainty only.

Why not send every screenshot to a multimodal AI model? Cost, latency, speed, and auditability can be poor for a fixed UI. First benchmark ROI/CV and use OCR or other models only where labeled evidence shows they are needed.

Does 100 tables mean 100 Docker containers? No. Measure permitted browser sessions, rendering behavior, CPU/RAM, capture rate, and coverage before choosing a worker count.

How can we know rounds were not missed? A trustworthy source sequence/ID, heartbeat, timestamps, gap detection, and comparison with independently labeled data help. Without a comparison source, coverage is uncertain; do not claim 100%.

Which database does the agent use? PostgreSQL 17 is the default. The old SQLite file is used only by the one-time, read-only migration tool. A database upgrade does not make screenshot interpretations trustworthy.

Why schedule a report if the agent runs continuously? Observation and daily report generation are different jobs. The report job gathers already verified historical data.

Can the agent include other players' stakes? Only if a permitted source explicitly provides accurate, defined stake data. Roadmap colors cannot reveal total stakes.

Can an official export replace OCR? Yes, if the provider authorizes it. A documented export may be a stronger source of round identity and final results than screen parsing.

Which files enable automatic confirmation? An approved adapter produces TableEvidence under agent/site_adapter.py's contract. agent/coordinator.py runs the conservative pipeline and agent/live_store.py persists validated candidates, evidence, gaps, and conflicts. The structured export and configurable DOM adapters provide input formats; the shipped DOM selectors have not been validated against a named website. Enable the trusted-ID gate only after a held-out source review.

12 — Specification for the Baccarat analytics agent

Purpose and boundary

Collect verified historical results from multiple Baccarat tables, measure data coverage, aggregate by day and table, and produce traceable reports. The output is descriptive statistics. There is no betting action, game control, betting signal, or guarantee about the next round.

Current state: this repository is a prototype. Preview, marker monitoring, a configurable DOM reader, structured export replay, transactional reviewed CSV import, PostgreSQL, daily Excel, a separate verified-round 07:00–21:00 red-streak Excel report, a conservative FSM, and a source coordinator with evidence and checkpoint storage have code. It lacks a validated adapter for a named real website, measured coverage, managed session lifecycle, and production soak results.

Inputs, users, and outputs

Group	Definition
Inputs	Authorized UI or export; captures from a screen the operator may observe; independently reviewed CSV
Users	Operator, data reviewer, report reader
Current outputs	PostgreSQL observations, confirmed_rounds, observation_gaps; five-sheet Excel
Target outputs	Evidence-backed rounds, per-table quality/coverage, and conflict/unknown lists
Time	Store UTC; current daily report boundary is Asia/Ho_Chi_Minh

Functional requirements

ID	Requirement	Current state	Acceptance condition
F01	Assign the correct table_id to each observation	Partial	Reordered cards never assign results to a different table; mismatch becomes UNKNOWN/GAP
F02	Keep frames separate from rounds	Implemented	100 identical frames add zero confirmed rounds
F03	Extract phase, ID, final result, and revision from a real source	Configurable DOM reader and structured export adapter; no named source validated	Evidence and tests exist for each source/layout
F04	Confirm each round once	Structured source coordinator and durable writer implemented; real source trust pending	Independent frames, trusted ID, consistent final result, no restart duplication
F05	Mark missing or unknowable data	Structured source gaps, restart, stale and missing-table paths implemented; real page behavior unvalidated	Outage, stale frame, ROI shift, and shoe change are recorded and break continuity
F06	Aggregate outcomes, streaks, and Wilson 95%	Implemented	Confirmed only; by table/day; Tie/GAP breaks strict streak
F07	Show provenance and quality	Partial	Synthetic/real source, confirmed count, gaps, unknown, and coverage are visible

ID	Requirement	Current state	Acceptance condition
F08	Recover and run on a schedule	Checkpoint and continuous DOM loop exist; health, managed session, soak, and report scheduling remain	Backoff, heartbeat, checkpoint, and idempotent daily report
F09	Handle conflict	Structured source conflicts quarantined; manual CSV conflicts raise	Never overwrite; queue both payloads and evidence for review
F10	Count verified red runs that first reach 13 in the configured 07:00–21:00 window	Partial: stored confirmed rounds and explicit mapping; no real-source validation or coverage ledger	Verified color mapping; one event per maximal run at its 13th result; per-table counts, total, evidence, and coverage; boundary and gap cases pass

Target data flow

```
Authorized source → capture → frame envelope and quality checks
                          ├→ ROI/marker observations → PostgreSQL, never a round
                          └→ source adapter → TableEvidence → per-table FSM
                                                ├→ UNKNOWN/GAP → quality log
                                                └→ VerifiedResult → evidence and validation
                                                                  → single writer → confirmed_rounds

confirmed_rounds + gaps + coverage → analytics → Excel/read-only API
```

The source adapter and confirmed_round write path are a trust boundary. No marker-count shortcut is allowed. Use one adapter per approved source and versioned layout; do not assume one heuristic works across sites.

Components to implement

1. Capture and clock: frame UTC time, monotonic sequence, viewport, source ID, URL/page state; detect duplicate/stale frames, page errors, and logout. Database insertion time is not automatically round finish time.
2. Identity and layout: verify a displayed table name and layout version before cropping. A mismatch blocks that table's confirmation.
3. Source adapter: produce TableEvidence with field-level provenance and quality. High OCR confidence alone does not make round_id_trusted true.
4. Validator: one FSM per table, building on ConservativeRoundValidator; add freshness, sequence, shoe/reset, and restart recovery.
5. Event store: save confirmed results, gaps, evidence metadata, and conflicts atomically or through one writer queue. Include source/shoe namespace if IDs can collide.
6. Analytics and report: read a consistent snapshot and display confirmed, unknown, gap, and coverage per table. Version metric definitions if semantics change.
7. Operations: health, structured logs, stale/low-coverage alerts, backup/restore, capture retention, and access control.

Business invariants

- Each confirmed result has a stable (source, table, round) identity, Banker/Player/Tie outcome, UTC finish time or documented estimate, and retrievable evidence.
- Conflicting results for one key require review; never silently replace the old row.

- Missing ID, phase, final outcome, or freshness means UNKNOWN. Lost continuity means GAP, not an inferred round.
- Rates use confirmed rounds only. Show coverage so missing data is not mistaken for zero games.
- Tie breaks the current strict streak. Different roadmap semantics need a separately named/versioned metric.
- Synthetic data must use a separate database.
- Never interpret a site's red color as Banker or Player without a validated mapping for that source/layout. Count 13-red events only from verified results, independently by table, using the window rules in chapter 07.

Decisions before a real adapter

Complete docs/15-source-survey.md for each source: access permission/rate, ID stability across shoes, where final results appear, Tie rules, UI latency, viewport/layout, number of simultaneously visible tables, and evidence retention. If no trustworthy source ID exists, use a local sequence only after continuity and restart recovery have been demonstrated. Otherwise keep manual verification.

Delivery milestones

Milestone	Reviewable output	Exit gate
M0 — current prototype	Synthetic preview/CSV/structured export/Excel, red-window report, and 30 tests, fourteen requiring PostgreSQL and one Chromium	Runs on target machine; team understands data is synthetic and coverage is unknown
M1 — one real table	Adapter, labeled corpus, evidence viewer	Held-out criteria in chapter 14 pass
M2 — 3–6 tables	Worker, checkpoint, gap/conflict, coverage report	24–72 hour soak and per-table comparison
M3 — scale	CPU/RAM/latency benchmark and worker groups	Per-table quality does not decline; backup/restore and alerts pass

Do not promise dates for M1–M3 until an authorized source has been surveyed.

13 — Data contracts and integration points

Current versus target interfaces

agent/round_validator.py takes TableEvidence(table_id, frame_id, phase, source_round_id, final_result, roadmap_revision, round_id_trusted) and returns VerifiedResult(table_id, source_round_id, result, evidence_frames) or None. agent/site_adapter.py defines CapturedFrame, ExtractedEvidence, and a SiteAdapter protocol. agent/pipeline.py runs the FSM and emits ConfirmedCandidate with two evidence references. The coordinator and durable writer persist approved structured candidates.

The legacy monitor-browser path calls vision.analyze_frame() and storage.record_observations() only. The newer monitor-dom and replay-export paths call StructuredExportAdapter, CandidatePipeline through SourceCoordinator, and live_store.save_candidate(). The CLI CSV importer remains the manual reviewed path. The configurable DOM selectors are placeholders, so this is a running generic pipeline but not a validated named-source API.

Frame envelope

A target capture record should include:

```
{
  &amp;quot;schema_version&amp;quot;: 1,
  &amp;quot;source_id&amp;quot;: &amp;quot;approved-provider-01&amp;quot;,
  &amp;quot;capture_id&amp;quot;: &amp;quot;uuid-or-monotonic-id&amp;quot;,
  &amp;quot;captured_at_utc&amp;quot;: &amp;quot;2026-10-19T05:00:00.000Z&amp;quot;,
  &amp;quot;received_at_utc&amp;quot;: &amp;quot;2026-10-19T05:00:00.120Z&amp;quot;,
  &amp;quot;viewport&amp;quot;: {&amp;quot;width&amp;quot;: 1536, &amp;quot;height&amp;quot;: 864, &amp;quot;scale&amp;quot;: 1},
  &amp;quot;layout_version&amp;quot;: &amp;quot;lobby-v1&amp;quot;,
  &amp;quot;page_state&amp;quot;: &amp;quot;ACTIVE&amp;quot;,
  &amp;quot;evidence_ref&amp;quot;: &amp;quot;private://capture/123&amp;quot;,
  &amp;quot;image_sha256&amp;quot;: &amp;quot;64-hex-characters&amp;quot;
}
```

capture_id is unique within the source. captured_at_utc is frame time, not automatically finished_at_utc. A hash may detect repetition but does not prove a new round. evidence_ref should point to a minimal protected capture, not an account screenshot or token-bearing URL. LOGIN, ERROR, STALE, or UNKNOWN page state blocks confirmation.

Per-table evidence

Field	Rule
source_id and table_id	Verify against the visible table label; never infer from card order alone
frame_id	Stable frame identity; the two RESULT frames must differ
phase	BETTING, DEALING, RESULT, or UNKNOWN before entering the FSM
source_round_id	Normalize and include shoe namespace if the source resets IDs

Field	Rule
round_id_trusted	True only after source/parser validation; OCR confidence alone is insufficient
final_result	BANKER, PLAYER, TIE, or null; only the published final outcome
roadmap_revision	Versioned roadmap fingerprint, not merely total marker count
captured_at_utc and evidence_ref	Carried by the envelope because the original TableEvidence dataclass lacks them

An adapter should preserve unreadable fields as UNKNOWN with a reason, never manufacture a round ID from timestamp or marker position. If published result and roadmap disagree, route to review. Document Tie behavior for the exact road being parsed.

Target confirmation flow

```
for each fresh frame and each verified table mapping:
    adapter.extract(frame) → evidence or UNKNOWN
    if source/layout/session is unhealthy:
        validator.gap(); persist GAP; continue
    candidate = validator.accept(evidence)
    if candidate is absent:
        persist observation/quality only; continue
    require both evidence frames, ordered, same source/table/round/result
    apply a documented finish-time policy
    transaction: evidence metadata + confirmed round + checkpoint
    exact duplicate → no-op; conflict → quarantine and alert
```

ConservativeRoundValidator.gap() resets memory only. The coordinator must persist the gap separately. emitted_ids in RAM is not restart protection; a durable unique key is the final guard. If a checkpoint cannot restore safe context, start UNKNOWN and wait for a new full cycle.

CandidatePipeline still stops at a candidate. SourceCoordinator and live_store now provide the separate durable writer, evidence links, checkpoint, duplicate-frame guard, idempotent source gaps, and conflict quarantine. Its observed_result_at_utc is the second result frame time with an explicit policy label, not a guaranteed source finish time.

Target time-window streak result

The read-only red_window_data() report now requires an exact stored source and a red_outcome mapping supplied by the caller and orders stored verified rounds per table. It returns the report date, source, time zone, 07:00–21:00 window, per-table counts, total count, first/13th round keys and finish times, and available evidence references for each event. The count is assigned to the window containing the 13th result, even when the run began before 07:00. A run that reaches 13 once cannot create another event by continuing. Recorded gaps break the run. The current schema has no source namespace in its legacy unique key and no coverage ledger. The structured writer stores source-prefixed round keys and evidence references, but the report still rejects a table ID shared by multiple sources and marks coverage UNKNOWN; source/layout mapping and complete coverage remain target requirements. See docs/07-statistics-and-excel.md and acceptance cases A13–A16.

Storage migration plan

Target table	Required additions	Purpose
captures	capture ID, source, UTC, layout, hash, evidence reference, deletion deadline	Frame trace and stale detection
table_evidence	capture/table, phase, round ID, result, revision, parser version, quality reason	Audit each decision
confirmed_rounds	source ID, shoe ID if needed, first/last evidence, validator version, finish-time policy	Prevent cross-source collisions and reproduce decisions
observation_gaps	start/end, reason code, source, unique event key	Downtime measurement and idempotent import
conflicts	round key, old/new payload, evidence, review state	Preserve disagreements
worker_checkpoints	source/table, last capture and round, phase, revision, UTC	Controlled recovery

PostgreSQL version 1 is defined in migrations/001_initial.sql. Migration 003 adds source evidence, unique source gaps, conflicts, processed frames, and checkpoints. The legacy confirmed unique key is still (table_id, round_id); the new writer encodes source in its stored round key. Manual gap imports are still not deduplicated. Before changing the key, back up the database, map existing rows to an explicit source namespace, add a new numbered migration, and test rollback/restore. Do not edit an applied migration or a production database ad hoc.

Time and ordering

captured_at_utc is when a frame was taken; finished_at_utc is the source's round-end time if available, otherwise a labeled estimate. Verify frame order with a monotonic sequence as well as wall-clock time. Late or reordered frames must not move the FSM backward. If there is no source finish time, report the timing uncertainty, especially around local midnight. The current daily_data() reports one Ho Chi Minh City day and does not calculate a cross-midnight streak.

Import and report idempotency

The current CSV importer requires verified=YES, reviewer, source, method, an offset-aware time, and valid outcome. The whole file now runs in one PostgreSQL transaction, so an error rolls back every row. Future work should add staging, import_batch_id, and per-row status. Gap import still needs a unique event key.

A report rerun currently overwrites the same workbook. Automated publishing needs report_runs(date, timezone, watermark, version, checksum, status) to prevent duplicate sends and reproduce a snapshot. Show each table's source and coverage; missing tables are not zero-round tables.

14 — Testing and acceptance for the target agent

Two test layers

Prototype: run python3 -m unittest discover -s tests -v with dependencies installed, or docker compose run --rm test. The 30 tests cover the conservative FSM, structured candidate-to-storage path, frame replay, source conflict quarantine, restart GAP, PostgreSQL deduplication, transactional rollback, one-time SQLite migration, report export, red-window boundaries and mixed-source rejection, strict streaks, gaps, local-day boundary, synthetic ROI, and a Chromium DOM-reader smoke test. Fourteen tests require PostgreSQL and one requires Chromium; Compose runs both. They do not establish accuracy on a real website.

Real adapter: create an independently labeled corpus for every source/layout. Preserve permitted captures, timestamps, table labels, phase, round ID, final result, shoe/reset, gaps, and unreadable states. Hold out entire sessions/days/tables from calibration.

Ground truth workflow

1. Record source ID, UI version, viewport, UTC interval, table, permission, file hash, and deletion policy in a manifest.
2. Have two people label finished rounds and unobservable intervals independently; resolve disagreements by reviewing evidence.
3. Label insufficient evidence UNKNOWN. Do not use the agent's own output as truth.
4. Cover Banker, Player, Tie, pair overlay, shoe reset, reordered cards, scroll, logout, lag, stale frames, network loss, restart, and the Ho Chi Minh City midnight boundary.
5. Split calibration and held-out data by session, not by random adjacent frames of the same round.

The earlier 200–500 round suggestion across 3–6 tables is a starting corpus, not proof that rare errors are absent. Report metric confidence intervals and expand the corpus according to observed risk.

Required metrics

Metric	Meaning	Breakdown
Confirmation precision	TP/(TP+FP), matching table, ID, and outcome	Per table/source/layout and overall
Recall	TP/(TP+FN) over ground-truth ended rounds	Separate detector misses, gaps, and absent source signals
Duplicate rate	Extra repeated confirmed rows / emitted rows	After replay and restart
Conflict rate	Same key with different payload	Evidence and review outcome
Coverage	Observable time/rounds divided by in-scope time/rounds	Per table and hour; never extrapolate to hidden tables
Gap detection	Reported true gaps and detection delay	Outage, stale frame, layout change
Latency	Capture → confirm → report, p50/p95/p99	By table count and hardware

A correct color on the wrong table counts as both a false positive and a false negative. UNKNOWN is not confirmed. High precision from confirming almost nothing is insufficient without recall and coverage.

Minimum integration matrix

ID	Scenario	Expected result
A01	Replay 100 identical frames	Zero new rounds
A02	Stable BETTING → DEALING → RESULT with trusted ID	One round with two evidence frames
A03	Banker changes to Player between RESULT frames	No confirmation; quality/conflict event
A04	Tie overlay without new marker	One Tie only if final source result confirms it; strict streak breaks
A05	Round ID resets after shoe change	No key collision; correct gap/rebaseline
A06	Disconnect or stale frame	UNKNOWN/GAP; no streak across missing interval
A07	Table card reorders or ROI shifts	Block wrong mapping
A08	Restart while RESULT is displayed	No duplicate; recover safely or await a new cycle
A09	Two writers process one key	One row or a reviewed conflict, with evidence preserved
A10	23:59:59–00:00:01 local boundary	Correct day by documented time policy
A11	CSV has an error midway	Production importer rolls back the whole batch
A12	Report reruns at one watermark	Same checksum and no duplicate delivery
A13	Twelve red results before 07:00, then the 13th at 07:00; run continues to 20	One reached-13 event inside the window, with no repeat for results 14–20
A14	The 13th red result at 21:00 or a gap before the 13th	Zero in-window reached-13 events; gap breaks continuity
A15	Two tables each first reach 13 inside the window	One event per table and two in the aggregate; no cross-table streak
A16	Site red mapping unverified or coverage incomplete	No asserted red-streak total; show unknown mapping or incomplete coverage

Suggested release gate

For each source/layout/table, approve a held-out report showing at least 99% confirmation precision and zero duplicate confirmed rows, with recall and coverage meeting thresholds set before evaluation. Review all conflicts. No confirmed round may come from marker count alone. The 99% figure was a proposed criterion in the original testing guide, not a measured prototype result. Include sample sizes and uncertainty; gather more data if the sample is too small.

Before 24/7 operation, run a 24–72 hour soak including deliberate restart and network loss, then check backup/restore, report latency, and each table's quality. Do not scale table count if p95 latency causes missed rounds or coverage falls. Rerun regression corpus tests for every adapter release and store parser/validator versions.

Acceptance package

Deliver a corpus manifest, labeling protocol, resolved disagreements, A01–A16 results, metrics by source/layout/table, false positive/negative cases, masked evidence links, source configuration, viewport, verified red mapping, Tie semantics, time policy, soak results, backup/restore results, and a named approval decision.

15 — Source survey template before writing an adapter

Copy this template into a separate private record for each authorized website or export. Do not place passwords, cookies, tokens, balances, or unmasked personal screenshots in the repository. Leave unknown answers as NOT DETERMINED; do not guess.

A. Authorization and scope

Question	Answer
Source name and responsible owner	NOT DETERMINED
Permission for automated observation, allowed rate, and retention rules	NOT DETERMINED
Permitted official API or export, if any	NOT DETERMINED
Tables and hours in scope	NOT DETERMINED
Screenshot access, masking, and deletion policy	NOT DETERMINED

B. UI and identity

Question	Answer
Page type and permitted session mechanism	NOT DETERMINED
Viewport, zoom, scale, and layout version	NOT DETERMINED
Visible table label, reordering, scrolling, and lazy loading	NOT DETERMINED
Round ID location and whether it resets per shoe/session	NOT DETERMINED
Shoe/session ID and source finish timestamp availability	NOT DETERMINED
BETTING/DEALING/RESULT signals and their display duration	NOT DETERMINED
Final Banker/Player/Tie source and update delay	NOT DETERMINED
Roadmap type and Tie/pair/reset/scroll semantics	NOT DETERMINED
Published final outcome represented by red in this exact source/layout; evidence for the mapping	NOT DETERMINED
Reporting time zone, 07:00–21:00 coverage, and treatment of results at the boundaries	NOT DETERMINED

C. Samples to collect

Capture at least one permitted, labeled example of a normal round, Banker, Player, Tie, shoe change, disconnect, logout, reordered card, full/scrolled roadmap, repeated frame, and theme/layout change. Record UTC time, table ID, viewport, file hash, and reviewer. Mask account information before sharing captures.

D. Feasibility decision

Decision	Answer
Can the adapter automatically verify rounds? Which evidence is sufficient?	NOT DETERMINED
Without a trusted ID, can continuity and restart recovery be demonstrated?	NOT DETERMINED

Decision	Answer
Which tables/hours remain observation-only or manually reviewed?	NOT DETERMINED
Parser version, approver, and approval date	NOT DETERMINED

Use the survey to finalize TableEvidence fields in docs/13-integration-contracts.md and the corpus/tests in docs/14-acceptance.md.

16 — Python and dependency version policy

Status and scope

This is the documentation target for a production-oriented proof of concept. It records what the repository currently declares and the version controls still needed before a reproducible release. This chapter does not claim that a new dependency lock or container build has been verified.

Python and infrastructure baseline

Component	Documented baseline	Current repository state
Python runtime	CPython 3.12.x, same minor version on developer machines, CI, and containers	The Playwright image uses Ubuntu 24.04 Noble, whose default Python is 3.12; the Dockerfile does not pin a Python patch release
Browser image	mcr.microsoft.com/playwright/python:v1.55.0-noble	Fixed tag in Dockerfile; browser binaries match Playwright 1.55.0
PostgreSQL	Major version 17	postgres:17 in compose.yaml; the image tag can move to a newer 17.x patch release
Operating system	Ubuntu 24.04 LTS in the application image	Defined by the Playwright Noble image

Python 3.12 is the proposed project baseline because Ubuntu 24.04 defaults to it. The exact Python patch version must be recorded from the chosen image when an immutable image digest is selected. Do not assume a developer's python3 executable is 3.12 without checking it.

References: Ubuntu's Python version table, Playwright's Docker guidance.

Direct Python packages

The table is an inventory of every third-party package explicitly declared by this repository. The declared version column reproduces the dependency files; a range is not an exact installed version.

Package	Import or purpose	Declared version	File	Used by
Playwright	playwright, browser capture	==1.55.0	requirements.txt	Runtime, tests that exercise browser integration later
OpenCV, headless build	cv2, image decoding and ROI processing	==4.11.0.86	requirements.txt	Runtime, synthetic image tool, tests
NumPy	numpy, image arrays	>=1.26,<3	requirements.txt	Runtime, synthetic image tool, tests
PyYAML	yaml, table configuration	>=6.0,<7	requirements.txt	Runtime, synthetic image tool, tests
XlsxWriter	xlsxwriter, Excel reports	>=3.2,<4	requirements.txt	Runtime, report tests
Psycopg 3 with binary package	psycopg, PostgreSQL access	>=3.2,<3.4	requirements.txt	Runtime, database tests, legacy import
python-docx	docx, DOCX handbook generation	==1.2.0	docs-requirements.txt	Documentation build only
ReportLab	reportlab, PDF handbook generation	==4.4.4	docs-requirements.txt	Documentation build only

The psycopg[binary] extra also installs a matching psycopg-binary distribution. Other packages may pull in further indirect distributions; their exact names and versions belong in the future resolved lock, not in a guessed list.

The Python standard library supplies argparse, asyncio, collections, csv, dataclasses, datetime, hashlib, html, json, math, os, pathlib, re, sqlite3 (legacy import only), tempfile, textwrap, time, typing, unittest, uuid, zipfile, and zoneinfo. These modules do not belong in a pip requirements file. PostgreSQL, Chromium, and browser system libraries are external runtime components, not pip packages.

Reproducibility gap and release policy

The four packages with >= constraints can resolve to different versions on different build dates. Direct pins also do not pin their indirect dependencies. For that reason, the current dependency files are a version policy for the POC, not a complete production lockfile. The repository does not yet contain an exact, hash-checked lock of the full Python dependency graph. No exact transitive version should be inferred from this document.

Before a production-oriented release:

1. Choose an immutable application-image digest and PostgreSQL-image digest. Record the Python 3.12 patch version present in the application image.
2. Keep a short list of direct dependencies, resolve it under Python 3.12 on the target Linux architecture, and commit an exact lock containing every direct and indirect distribution and its hashes. Lock documentation-build packages separately from runtime packages.
3. Install from the committed lock in CI and in the application Dockerfile. Keep the Playwright Python package version equal to the browser image version.
4. Rebuild and run the unit, PostgreSQL integration, browser smoke, documentation-build, and security checks after every dependency update. Record the lock revision, image digests, and test result for each release.
5. Review PostgreSQL major upgrades separately, including backup and restore. Allow controlled patch updates only after validation.

These steps are release requirements, not a claim that the current POC has already completed them. Do not change requirements.txt, docs-requirements.txt, or Docker images solely by copying suggested numbers from documentation; resolve and test the complete graph first.

17 — Structured source integration and live DOM trial

Current boundary

The source-independent path is implemented: structured capture → StructuredExportAdapter → per-table SourceCoordinator → conservative phase validator → transactional PostgreSQL round/evidence writer → reports. A configurable DOM reader can populate the same path when an authorized page exposes explicit text fields. The repository still has no validated adapter for a named Baccarat website. config/real_dom.example.yaml contains placeholder selectors and must not be treated as a working site configuration.

The older monitor-browser and monitor-image commands remain marker-observation only. monitor-dom uses the structured path. Without --trusted-round-ids, it cannot confirm a round. With that flag, the operator asserts that the exact page, selectors, ID/reset semantics, phase and result labels, and evidence policy passed the source review in docs/15-source-survey.md and the held-out tests in docs/14-acceptance.md. The flag is a technical gate, not proof of accuracy.

Try the synthetic structured export

The file samples/structured_export_demo.jsonl contains four synthetic captures for one round. It has BETTING, DEALING, and two stable RESULT frames, a shoe ID, a round ID, a final BANKER result, and a changed roadmap revision. Use a disposable database or a separate Compose project:

```
docker compose run --rm replay-export-demo
```

Expected first result: three OBSERVED states and one INSERTED. Replaying the same file returns duplicate-frame statuses and no new round. This fixture does not establish any website's accuracy. The command-line equivalent is:

```
docker compose run --rm replay-export-demo python -m agent.cli replay-export \

  --input samples/structured_export_demo.jsonl \

  --source-id DEMO_STRUCTURED --tables S01 --trusted-round-ids
```

For a real authorized structured export, leave --trusted-round-ids off until the source identity and round semantics are independently validated. Each JSONL line follows the field contract in samples/STRUCTURED_EXPORT_SCHEMA.md. Each frame is committed separately, so a later invalid line does not roll back prior valid frames. Use a fresh database for a new source or layout during evaluation.

Configure an authorized DOM page

1. Complete docs/15-source-survey.md and obtain permission to observe the source at the intended capture rate.
2. Copy config/real_dom.example.yaml to a private configuration. Set the real URL, fixed viewport, stable table selectors, visible table label, phase, round ID, shoe ID if IDs reset, final result, and roadmap revision. root must uniquely select the intended table. The visible label must equal the configured table ID. Set phase/result maps for the source's exact text.
3. Run a short observation-only trial and inspect the cropped table evidence under output/dom_evidence/:

```
docker compose run --rm monitor-dom python -m agent.cli monitor-dom \

  --config config/my_source_dom.yaml --iterations 30 --interval 5
```

4. Compare every extracted field against an independently labeled capture corpus. Include Tie, shoe reset, login/error states, table reordering, missing selectors, and page delays. Store the corpus and review decision outside the repository if it contains private data.
5. Only after the source passes the acceptance gate, run with --trusted-round-ids. Use --iterations 0 for an open-ended worker. The worker records missing-table, adapter, stale-frame, restart, and capture-gap events

and fails closed when it cannot establish continuity.

The DOM reader only handles fields exposed as accessible text in one page context. Canvas-only content, cross-origin iframes, login/session workflows, and source-specific result behavior need a separate reviewed adapter. There is no generic safe way to infer these from a screenshot or marker count.

Persistence and reporting

Migration 003 adds source-round evidence, idempotent source gaps, processed frame keys, checkpoints, and quarantined conflicts. The writer stores a source-prefixed key in the existing confirmed_rounds table so a second source's raw round ID cannot overwrite it. A shoe ID must be included when the source reuses round IDs by shoe. On restart the coordinator records a GAP and requires a new BETTING → DEALING → RESULT cycle. It does not infer rounds missed while stopped.

source_round_evidence keeps the two frame IDs and evidence references for confirmed rounds. The 07:00–21:00 red report includes these references when available. Existing reviewed CSV rows may lack them. Observation coverage remains UNKNOWN because the project has no independently verified source ledger or measured coverage ratio. The current observation_gaps table is keyed only by table, so a database with multiple sources reusing the same table ID is not supported by the red report; it rejects that case. Keep real and synthetic sources in separate databases.

Review the source-specific red-to-outcome mapping before running:

```
docker compose run --rm report python -m agent.cli report-red-window \
  --date 2026-10-19 --source DEMO_STRUCTURED --red-outcome BANKER
```

This fixture has only one round, so its reached-13 count is zero.